

Innovatech Solutions

Informe de Pruebas Unitarias
Cobertura y Metricas | DuocUC 2026

1. Resumen Ejecutivo

El proyecto Innovatech Solutions cuenta con 82 pruebas automatizadas implementadas con Jest, ts-jest y Supertest, distribuidas en los 4 servicios del sistema. Las pruebas cubren tres tipos: unitarias (65 tests), integracion (8 tests) y end-to-end (8 tests + 1 smoke). Todas las pruebas se ejecutan sin necesidad de Docker ni base de datos, utilizando mocks de Prisma y axios.

2. Distribucion de Pruebas por Servicio

Servicio	Tipo	Suite	Tests
BFF Gateway	Unitaria	auth.middleware.test.ts	6
BFF Gateway	Integracion	auth.routes.test.ts	8
BFF Gateway	End-to-End	flow.test.ts	8
ms-projects	Unitaria	project.factory.test.ts	7
ms-projects	Unitaria	project.service.test.ts	16
ms-tasks	Unitaria	task.factory.test.ts	7
ms-tasks	Unitaria	task.service.test.ts	11
ms-teams	Unitaria	team.factory.test.ts	6
ms-teams	Unitaria	team.service.test.ts	12
		TOTAL	82

3. Metricas de Cobertura

La cobertura se midio con Jest --coverage. Los resultados por servicio son:

3.1 BFF Gateway

Archivo	% Stmts	% Branch	% Funcs	% Lines
auth/auth.routes.ts	100%	100%	100%	100%
config/jwt.config.ts	100%	100%	100%	100%
middleware/auth.middleware.ts	78.94%	50%	33.33%	76.47%
routes/projects.routes.ts	59.52%	100%	57.14%	58.53%
routes/tasks.routes.ts	52.77%	100%	50%	51.42%

services/projects.service.ts	73.07%	44.44%	47.61%	80%
services/tasks.service.ts	71.05%	33.33%	47.05%	81.81%
TOTAL	69.56%	54.16%	49.09%	72.25%

3.2 ms-projects

Archivo	% Stmts	% Branch	% Funcs	% Lines
factories/project.factory.ts	100%	100%	100%	100%
services/project.service.ts	100%	94.44%	100%	100%
TOTAL	100%	96.15%	100%	100%

3.3 ms-tasks

Archivo	% Stmts	% Branch	% Funcs	% Lines
factories/task.factory.ts	100%	100%	100%	100%
services/task.service.ts	100%	100%	100%	100%
TOTAL	100%	100%	100%	100%

3.4 ms-teams

Archivo	% Stmts	% Branch	% Funcs	% Lines
factories/team.factory.ts	100%	100%	100%	100%
services/team.service.ts	100%	100%	100%	100%
TOTAL	100%	100%	100%	100%

3.5 Resumen General de Cobertura

Servicio	% Stmts	% Branch	% Funcs	% Lines	Resultado
BFF Gateway	69.56%	54.16%	49.09%	72.25%	Supera 60%
ms-projects	100%	96.15%	100%	100%	Supera 60%
ms-tasks	100%	100%	100%	100%	Supera 60%
ms-teams	100%	100%	100%	100%	Supera 60%

4. Tipos de Prueba Implementados

4.1 Pruebas Unitarias (65 tests)

Testean las capas internas de cada microservicio de forma aislada. Las Factories se testean directamente verificando validaciones y valores por defecto. Los Services se testean con Prisma

mockeado mediante `jest.mock("@prisma/client")`, lo que permite verificar toda la logica de negocio sin necesidad de base de datos. La prueba mas importante valida la regla de negocio critica: no se puede completar un proyecto con tareas pendientes.

4.2 Pruebas de Integracion (8 tests)

Testean el flujo de login completo del BFF usando Supertest contra Express real. Verifican autenticacion exitosa con bcrypt para los 3 usuarios, rechazo de credenciales invalidas, validacion de campos obligatorios, y generacion correcta del JWT.

4.3 Pruebas End-to-End (8 tests)

Simulan el flujo completo de negocio en el BFF: login, proteccion de rutas, creacion de proyecto, creacion de tarea, intento de completar proyecto que falla por tarea pendiente, completar la tarea, completar el proyecto exitosamente, y verificacion del fallback del Circuit Breaker cuando un microservicio cae.

5. Herramientas Utilizadas

Herramienta	Version	Proposito
Jest	29/30	Framework de testing y runner
ts-jest	29	Transformador TypeScript para Jest
Supertest	7	Testing HTTP contra Express
@types/jest	29/30	Tipos TypeScript para Jest
jest.mock()	Integrado	Mock de Prisma y axios

6. Como Ejecutar las Pruebas

En cada servicio:

```
npm test # Ejecutar tests
npx jest --coverage --forceExit # Ejecutar con reporte de cobertura
```

Las pruebas no requieren Docker ni base de datos. Se ejecutan en menos de 5 segundos por servicio. Los mocks de Prisma simulan la base de datos y los mocks de axios simulan la comunicacion entre microservicios.

7. Patrones de Diseno en las Pruebas

Las pruebas reflejan los patrones de diseno del proyecto. El patron Factory se testea verificando que cada Factory valide campos obligatorios y asigne estados por defecto. El patron Service Layer se testea mockeando Prisma para aislar la logica de negocio. El Circuit Breaker se testea simulando la caida de un microservicio y verificando que el fallback se retorne correctamente. El Middleware Chain se testea verificando que el verifyToken acepte tokens validos y rechace los invalidos.